

ПРАКТИЧЕСКАЯ РАБОТА № 14

Тема: Создание зашифрованных каналов с помощью OpenVPN

Программное обеспечение:

1. ОС Ubuntu Server 16.04

[https://drive.google.com/drive/folders/1cWpO8QwGVkmwYIRfAH5KZ7v3z2Ng4hCm?usp=drive link](https://drive.google.com/drive/folders/1cWpO8QwGVkmwYIRfAH5KZ7v3z2Ng4hCm?usp=drive_link)

2. Примечание: Работа выполняется на тестовом стенде, состоящем из 3-х виртуальных машин под управлением ОС Linux (используется Ubuntu Server 16.04, возможен другой дистрибутив, основанный на Debian).

Вопросы для обсуждения:

1. Какой криптографический протокол используется в OpenVPN?
2. OpenVPN в качестве транспортного протокола может использовать как TCP, так и UDP. В чем преимущество использования каждого?
3. В настройках OpenVPN присутствует выбор типа туннеля TUN или TAP. Чем они отличаются?
4. Что такое OpenVPN?
5. Для чего используется технология OpenVPN?
6. Назовите основной протокол, применяемый OpenVPN для передачи данных.
7. Какие виды сертификатов применяются в OpenVPN?
8. Что означает аббревиатура TLS в контексте OpenVPN?
9. Какие порты чаще всего используются для работы OpenVPN?
10. Где хранятся сертификаты и ключи для авторизации клиентов OpenVPN?
11. Как называется конфиг-файл сервера OpenVPN?
12. Нужно ли перезапускать службу OpenVPN после внесения изменений в конфигурационный файл?

Цель: изучить технологию шифрования сетевого трафика, сетевых и криптографических протоколов.

Основные теоретические сведения

1. OpenVPN

OpenVPN – свободная реализация технологии виртуальной частной сети (VPN) с открытым исходным кодом для создания зашифрованных каналов типа точка-точка или сервер-клиенты между компьютерами. Она позволяет устанавливать соединения между компьютерами, находящимися за NAT и сетевым экраном, без необходимости изменения их настроек.

Для обеспечения безопасности управляющего канала и потока данных OpenVPN использует библиотеку OpenSSL. Это позволяет задействовать весь набор алгоритмов шифрования, доступных в данной библиотеке. Также может использоваться пакетная аутентификация HMAC, для обеспечения большей безопасности, и аппаратное ускорение для улучшения производительности шифрования. Эта библиотека использует OpenSSL, а точнее протоколы SSLv3/TLSv1.2. OpenVPN используется в операционных системах Solaris, OpenBSD, FreeBSD, NetBSD, GNU/Linux, Apple Mac OS X, QNX, Microsoft Windows, Android, iOS.

OpenVPN предлагает пользователю несколько видов аутентификации:

- предустановленный ключ – самый простой метод;
- сертификатная аутентификация – наиболее гибкий в настройках метод;
- с помощью логина и пароля – может использоваться без создания клиентского сертификата (серверный сертификат всё равно нужен).

OpenVPN проводит все сетевые операции через TCP или UDP транспорт. TCP обеспечивает лучшую надёжность передачи данных, однако критикуется за бóльшие задержки по сравнению с UDP, который выигрывает в скорости за счёт отсутствия подтверждения доставки пакетов. Также возможна работа через большую часть прокси серверов, включая HTTP, SOCKS, через NAT и

сетевые фильтры. Сервер может быть настроен на назначение сетевых настроек клиенту. Например: IP адрес, настройки маршрутизации и параметры соединения. OpenVPN предлагает два различных варианта сетевых интерфейсов, используя драйвер TUN/TAP. Возможно создать туннель сетевого уровня, называемый TUN, и канального уровня – TAP, способный передавать Ethernet-трафик. Также возможно использование библиотеки компрессии LZO для сжатия потока данных. Используемый порт 1194 выделен Internet Assigned Numbers Authority для работы данной программы. Версия 2.0 позволяет одновременно контролировать несколько туннелей, в отличие от версии 1.0, позволявшей создавать только 1 туннель на 1 процесс.

Использование в OpenVPN стандартных протоколов TCP и UDP позволяет ему стать альтернативой IPsec в ситуациях, когда Интернет-провайдер блокирует некоторые VPN протоколы.

2. SSH

SSH (англ. Secure Shell – «безопасная оболочка») – сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP-соединений (например, для передачи файлов). Схож по функциональности с протоколами Telnet и rlogin, но, в отличие от них, шифрует весь трафик, включая и передаваемые пароли. SSH допускает выбор различных алгоритмов шифрования. SSH-клиенты и SSH-серверы доступны для большинства сетевых операционных систем.

SSH позволяет безопасно передавать в незащищённой среде практически любой другой сетевой протокол. Таким образом, можно не только удалённо работать на компьютере через командную оболочку, но и передавать по зашифрованному каналу звуковой поток или видео (например, с веб-камеры). Также SSH может использовать сжатие передаваемых данных для последующего их шифрования, что удобно, например, для удалённого запуска клиентов X Window System.

SSH – это протокол прикладного уровня. SSH-сервер обычно прослушивает соединения на TCP-порту 22. Спецификация протокола SSH-2 содержится в RFC 4251. Для аутентификации сервера в SSH используется протокол аутентификации сторон на основе алгоритмов электронно-цифровой подписи RSA или DSA, но допускается также аутентификация при помощи пароля (режим обратной совместимости с Telnet) и даже ip-адреса хоста (режим обратной совместимости с rlogin).

- Аутентификация по паролю наиболее распространена. При каждом подключении подобно https вырабатывается общий секретный ключ для шифрования трафика.

- При аутентификации по ключевой паре предварительно генерируется пара открытого и закрытого ключей для определённого пользователя. На машине, с которой требуется произвести подключение хранится закрытый ключ, а на удалённой машине – открытый. Эти файлы не передаются при аутентификации, система лишь проверяет, что владелец открытого ключа также владеет и закрытым. При данном подходе, как правило, настраивается автоматический вход от имени конкретного пользователя в ОС.

- Аутентификация по ip-адресу небезопасна, эту возможность чаще всего отключают.

Для создания общего секрета (сеансового ключа) используется алгоритм Диффи – Хеллмана (DH). Для шифрования передаваемых данных используется симметричное шифрование, алгоритмы AES, Blowfish или 3DES. Целостность передачи данных проверяется с помощью CRC32 в SSH1 или HMAC-SHA1/HMAC-MD5 в SSH2.

Для сжатия шифруемых данных может использоваться алгоритм LempelZiv (LZ77), который обеспечивает такой же уровень сжатия, что и архиватор ZIP. Сжатие SSH включается лишь по запросу клиента, и на практике используется редко.

Поддержка SSH реализована во всех UNIX-подобных системах, и на большинстве из них в числе стандартных утилит присутствуют клиент и сервер ssh. Существует множество реализаций SSH-клиентов и для не-UNIX ОС. Большую популярность протокол получил после широкого развития анализаторов трафика и способов нарушения работы локальных сетей, как альтернативное небезопасному протоколу Telnet решение для управления важными узлами.

Для работы по SSH нужен SSH-сервер и SSH-клиент. Сервер прослушивает соединения от клиентских машин и при установлении связи производит аутентификацию, после чего начинает обслуживание клиента. Клиент используется для входа на удалённую машину и выполнения команд.

Для соединения сервер и клиент должны создать пары ключей – открытых и закрытых – и обменяться открытыми ключами. Обычно используется также и пароль.

Задание к практической работе

Практическая часть работы состоит из 7 частей:

1. Настройка сетевых интерфейсов виртуальных машин.
2. Подключение к виртуальным машинам по SSH для дальнейшего конфигурирования.
3. Настройка маршрутизации между устройствами.
4. Настройка трансляции ip-адресов.
5. Создание OpenVPN-сервера.
6. Создание конфигурации OpenVPN-клиента.
7. Перенаправление трафика в защищенный туннель.

Работа выполняется на тестовом стенде, состоящем из 3-х виртуальных машинах под управлением ОС Linux (используется Ubuntu Server 16.04, возможен другой дистрибутив, основанный на Debian):

Client – имитирует удаленное устройство, которое необходимо подключить к внутренней сети предприятия через небезопасный канал связи.

Server – имитирует устройство внутри корпоративной сети, на которое необходимо установить VPN-сервер для безопасного подключения клиента.

Web-server – узел сети внутри корпоративной сети, не имеет прямого соединения с клиентом.

1. Настройка сетевых интерфейсов виртуальных машин

В настройках каждой виртуальной машины первым сетевым интерфейсом назначаем NAT (рисунок 1). Данный интерфейс будет использоваться исключительно для управления узлами по SSH.

Для того, чтобы сделать возможным подключение к виртуальной машине по SSH с машины-хоста (на которой запущены виртуальные машины), необходимо задать настройки «проброса портов» за NAT. Осуществляется это путем нажатия на кнопку «Port Forwarding» и внесение следующих настроек, указанных на рисунке 2.

Так как внутренний сетевой интерфейс машины-хоста (127.0.0.1) будет использован для обращения к нескольким виртуальным машинам одновременно, необходимо назначить разные порты для разных виртуальных машин. Значения портов указаны в таблице 1.

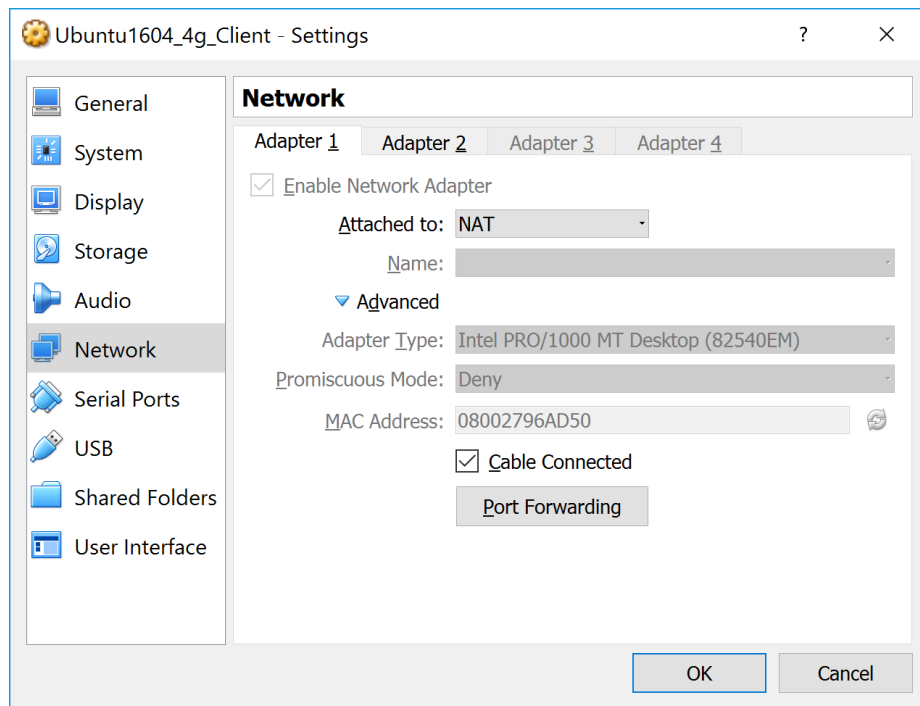


Рисунок 1 – Добавление первого сетевого интерфейса

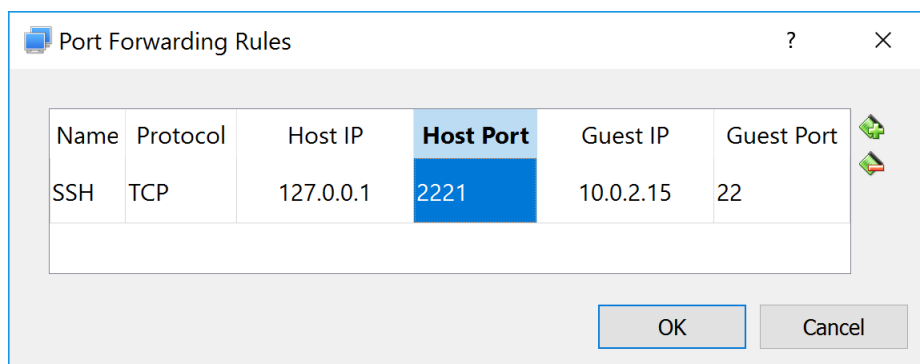


Рисунок 2 – Настройка проброса портов

Таблица 1 – Настройки проброса портов для виртуальных машин

Имя машины	Name	Host IP	Host Port	Guest IP	Guest Port
Client	SSH	127.0.0.1	2221	10.0.2.15	22
Server			2222		
Web-server			2223		

В случае, если в дальнейшем не удастся установить подключение, необходимо убедиться в том, что значение Guest IP для каждой виртуальной машины соответствует IP-адресу, полученному её первым сетевым

интерфейсом в процессе работы. Узнать, какие IP-адрес были присвоены сетевым интерфейсам машины можно в процессе её работы командой `ifconfig`.

Помимо установления подключения по SSH извне, необходимо обеспечить сетевое взаимодействие между виртуальными машинами. Целью данных настроек является получение сети с топологией, указанной на рисунке 3.

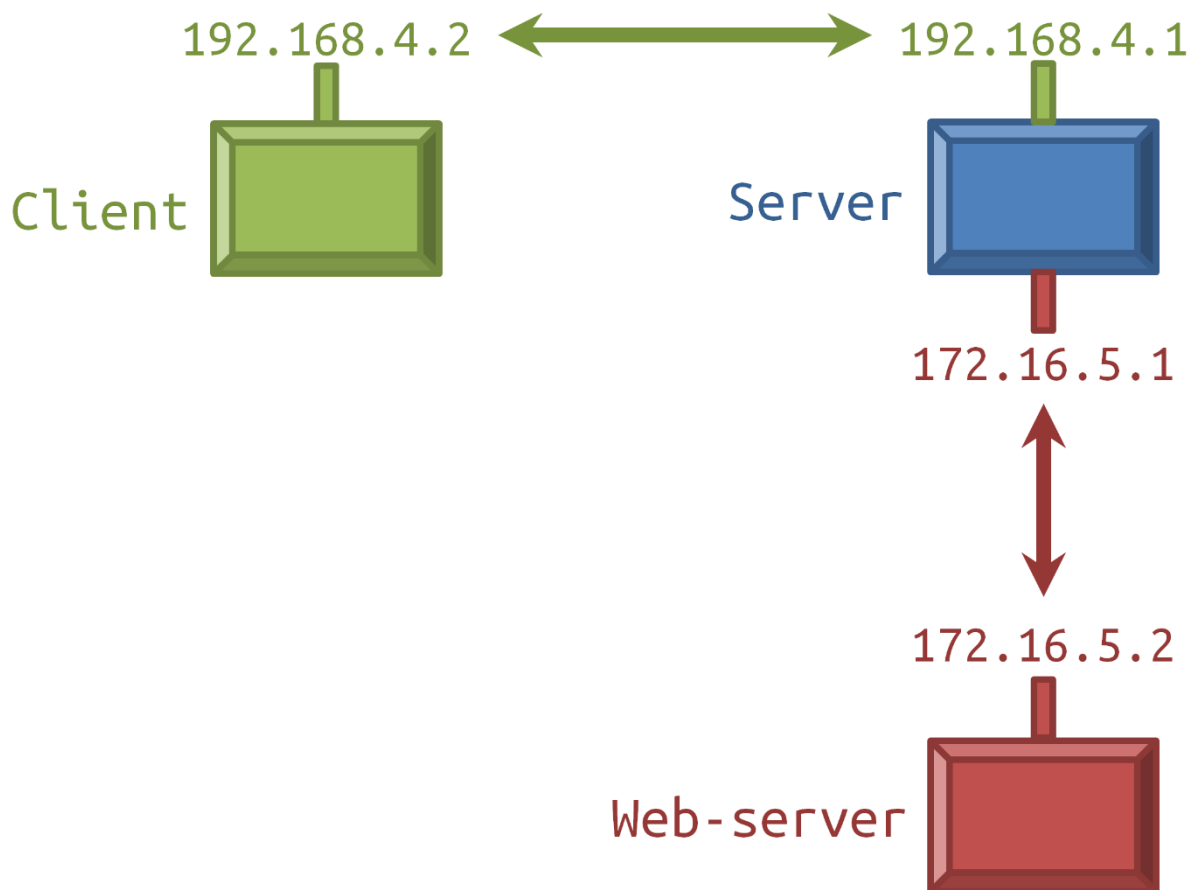


Рисунок 3 – Топология виртуальной сети

Для этого добавим следующие сетевые интерфейсы с типом Internal Network (рисунок 4):

Для **Client 1** сетевой интерфейс Internal Network:

адаптер №2 с именем сети net-1.

Для **Server 2** сетевых интерфейса Internal Network:

адаптер №2 с именем сети net-1;

адаптер №3 с именем сети net-2.

Для **Web-server 1** сетевой интерфейс Internal Network:
адаптер №3 с именем сети net-2.

Задание имени сети позволяет определить узлы в разные сегменты сети. Узлы с одинаковыми именами Internal Network будут помещены в одну подсеть. Подсеть net-1 будет иметь адрес 192.168.4.0/24, а подсеть net-2 – адрес 172.16.0.0/16.

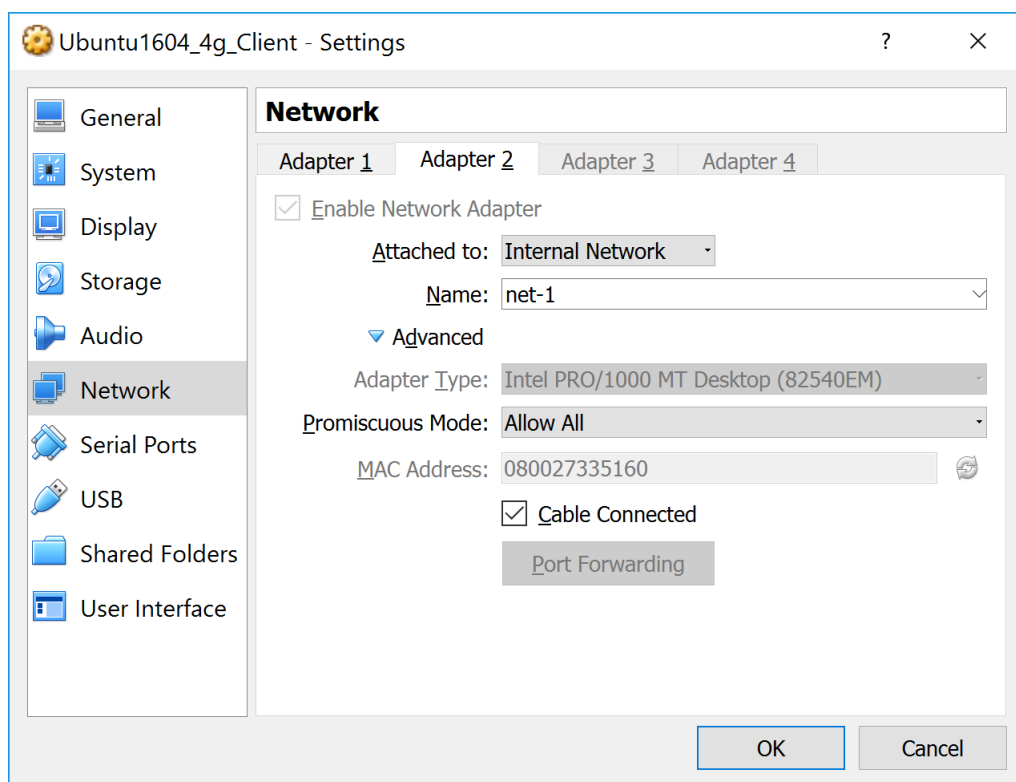


Рисунок 4 – Добавление адаптера внутренней сети

Для дальнейшей настройки необходимо запустить виртуальные машины. Целью данных настроек является присвоение сетевым интерфейсам виртуальных машин IP-адресов, указанных на рисунке 3.

Список существующих сетевых адаптеров (в том числе и неактивных, то есть со значением state==DOWN) можно узнать командой «*ip link*». Пример вывода команды «*ip link*» на сервере представлен на рисунке 5. На клиентской машине будет отсутствовать интерфейс enp0s9, а на веб-сервере – enp0s8.

```
user@server:~$ ip link
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
DEFAULT group default qlen 1
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
mode DEFAULT group default qlen 1000
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT
group default qlen 1000
4: enp0s9: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT
group default qlen 1000
```

Рисунок 5 – Список сетевых интерфейсов (цветом выделены имена интерфейсов)

Для того, чтобы узнать IP-адреса, присвоенные сетевым интерфейсам, используем команду «*ifconfig*» (рисунок 6).

```
enp0s3  Link encap:Ethernet HWaddr 08:00:27:b3:d0:09
        inet addr:10.0.2.15 Bcast:10.0.2.255 Mask:255.255.255.0
        ...
lo      Link encap:Local Loopback
        inet addr:127.0.0.1 Mask:255.0.0.0
        ...
```

Рисунок 6 – Вывод команды *ifconfig*

Как видим, не все интерфейсы имеют IP-адреса. Присвоим им адреса статически с помощью команды:

ifconfig <имя интерфейса> <IP-адрес> (рисунок 7).

```
user@server:~$ sudo ifconfig enp0s8 192.168.4.1
user@server:~$ sudo ifconfig enp0s9 172.16.5.1
user@server:~$ ifconfig
enp0s3  Link encap:Ethernet HWaddr 08:00:27:b3:d0:09
        inet addr:10.0.2.15 Bcast:10.0.2.255 Mask:255.255.255.0
        ...
enp0s8  Link encap:Ethernet HWaddr 08:00:27:81:c3:ce
        inet addr:192.168.4.1 Bcast:192.168.4.255 Mask:255.255.255.0
```

```
...
enp0s9 Link encap:Ethernet HWaddr 08:00:27:49:00:c9
      inet addr:172.16.5.1 Bcast:172.16.255.255 Mask:255.255.0.0
...
lo      Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
...

```

Рисунок 7 – Присвоение адресов сетевым интерфейсам

Аналогично присваиваем IP адреса для клиента и веб-сервера (смотри рисунок 3 и таблицу 2).

Таблица 2 – Адреса виртуальных машин

Имя машины	Имя интерфейса	IP
Client	enp0s8	192.168.4.2
Server	enp0s8	192.168.4.1
	enp0s9	172.16.5.1
Web-server	enp0s9	172.16.5.2

2. Подключение по SSH

Для удаленного конфигурирования часто используется SSH. Для его использования необходим SSH-сервер (устанавливается на управляемое устройство) и SSH-клиент (устанавливается на управляющее устройство). В ОС Linux часто ssh-клиент (а иногда и сервер) присутствует по умолчанию. В данной лабораторной работе на каждой виртуальной машине установлен openssh-server. В качестве клиента под ОС Windows используется Putty и WinSCP.

Выполненные в предыдущем разделе настройки сетевых интерфейсов (а именно, NAT с пробросом портов) позволяет подключаться к гостевым машинам используя порты хостовой машины.

Для запуска SSH-сервера используем команду инициализации:

sudo /etc/init.d/ssh restart

Для подключения с помощью Putty (рисунок 8) вводим адрес сервера и порт, на котором работает SSH (по умолчанию 22). Так как был настроен проброс портов, в качестве адреса указываем адрес хоста и порт из таблицы 1 (то есть, 2221 для подключения к машине «клиент», 2222 – для подключения к «серверу» и 2223 для подключения к «веб-серверу»).

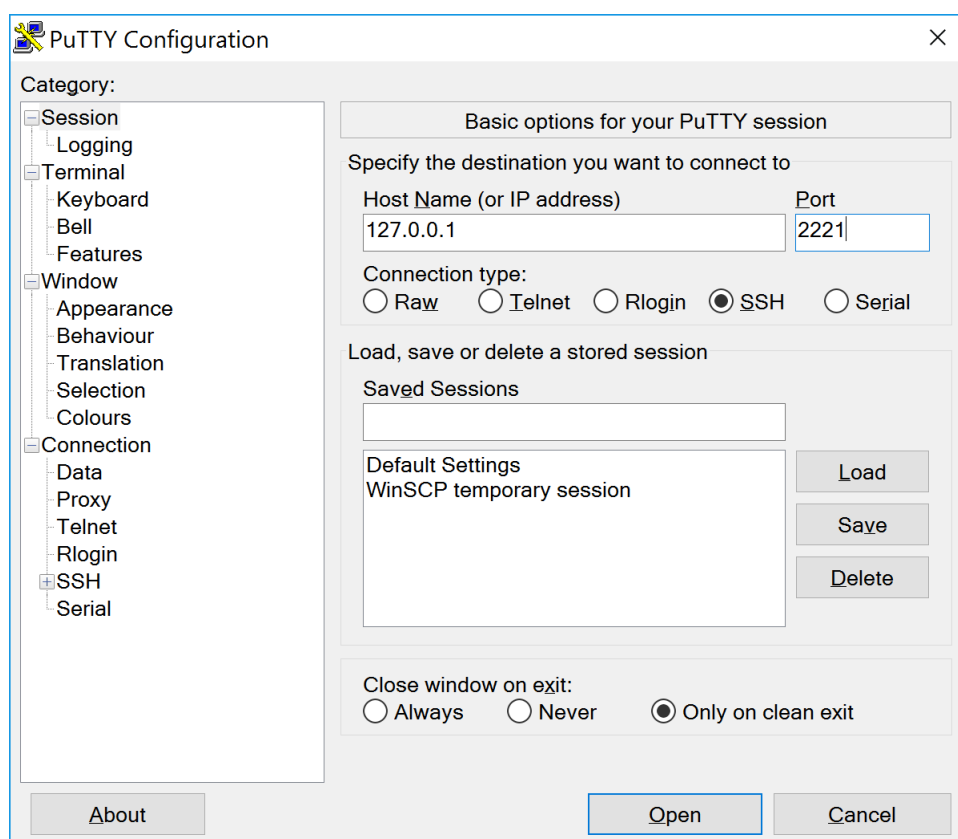


Рисунок 8 – Интерфейс SSH-клиента Putty

При подключении в первый раз возникает сообщение об отсутствии ключа в регистре. После подтверждения дальнейшие соединения к данному хосту шифруются по его ключу.

После установления соединения возникает окно терминала с предложением ввести имя пользователя и пароль (рисунок 9).

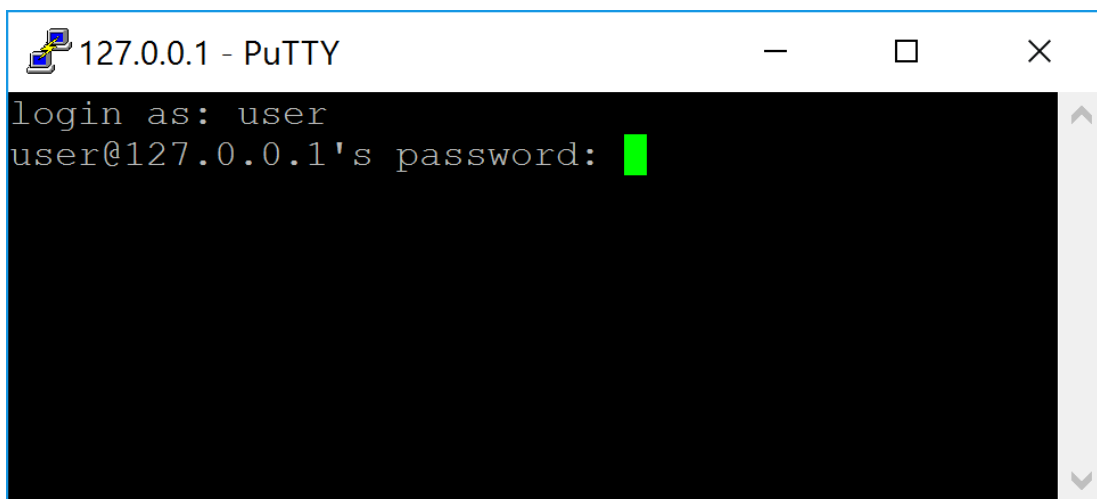


Рисунок 9 – Терминал Putty

После аутентификации в системе, с помощью данного терминала можно выполнять управление удаленной машиной.

Отметим некоторые особенности терминала Putty, облегчающие работу с ним:

- копирование производится с помощью выделения текста. То есть дополнительного нажатия Ctrl+C не требуется (тем более, что в Linux за этой комбинацией клавиш закреплено совершенно другое действие). Достаточно выделить фрагмент текста, и он окажется в буфере обмена;
- вставка в терминал осуществляется нажатием правой кнопки мыши;
- для создания нескольких терминалов, подключенных к одной машине, необходимо нажать правой клавишей на заголовке окна терминала и в появившемся меню выбрать Duplicate Session;
- автодополнение команд в терминале Linux осуществляется с помощью клавиши Tab.

Для передачи файлов на виртуальную машину и обратно можно использовать другой SSH-клиент – WinSCP (рисунок 10). После аутентификации и подтверждения ключа возникает окно, состоящее из двух частей. В левой показано содержимое файловой системы машины, на которой запущен WinSCP. А в правой – содержимое файловой системы удаленной

машины (рисунок 11). Копирование файлов можно осуществлять их перетаскиванием из одной части в другую.

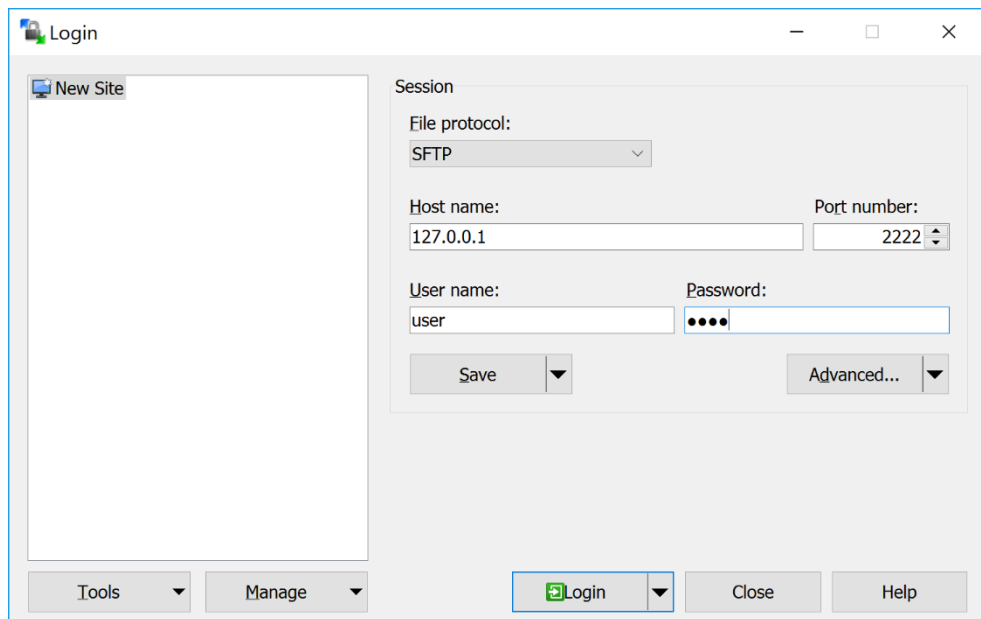


Рисунок 10 – Окно аутентификации WinSCP

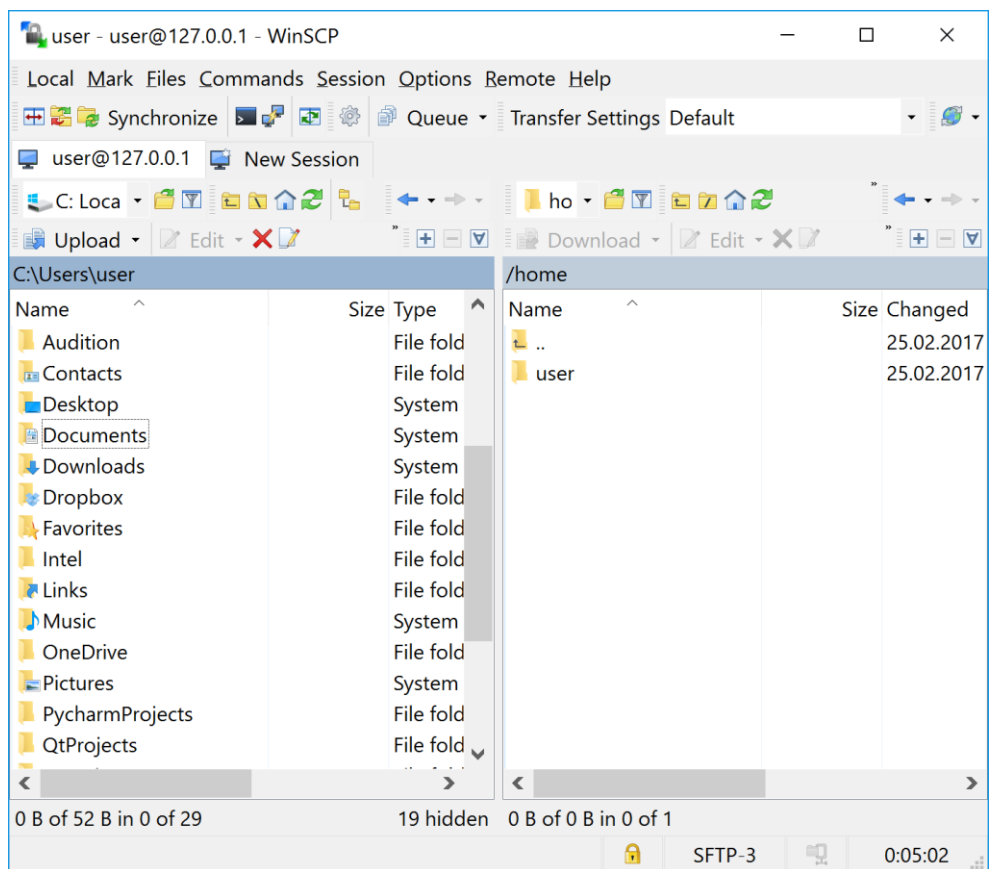


Рисунок 11 – Интерфейс WinSCP

3. Настройка маршрутизации

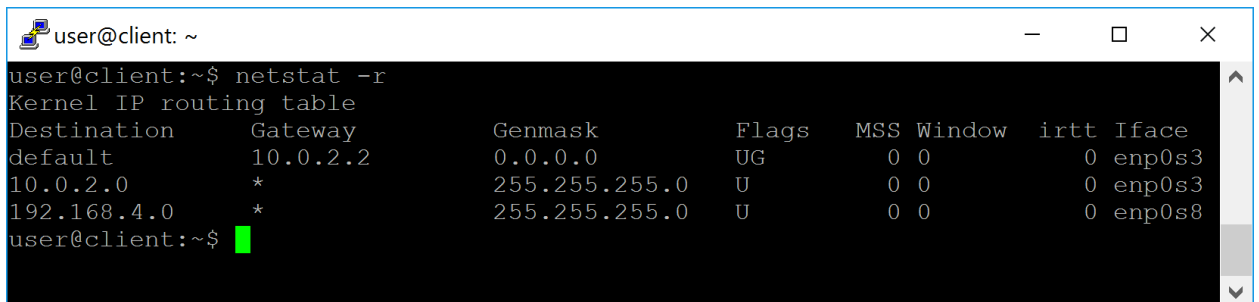
После присвоения сетевым интерфейсам IP-адресов согласно таблице 2, становится возможным сетевое взаимодействие Client ↔ Server и Server ↔ Web-server. Однако попытка клиента получить доступ к Web-серверу (например, отправив пинг командой «*ping 172.168.5.2*») не удастся по следующим причинам:

1. Клиент не знает, куда направлять пакеты с таким адресом.
2. Даже если он направит их на узел 192.168.4.1 (то есть, узлу Server), тот отбросит их, так как не является получателем.

Таким образом, данный раздел ставит следующую задачу:

1. Настроить узел client для отправки пакетов в сеть 172.16.0.0 по адресу узла Server.
2. Настроить узел Server для дальнейшей обработки таких пакетов.

Данные вопросы касаются маршрутизации (routing). Таблицу маршрутизации на узле можно просмотреть командой «*netstat -r*» (рисунок 12).



```
user@client: ~  
user@client:~$ netstat -r  
Kernel IP routing table  
Destination      Gateway          Genmask          Flags      MSS Window  irtt  Iface  
default          10.0.2.2        0.0.0.0          UG         0  0      0    enp0s3  
10.0.2.0         *              255.255.255.0    U         0  0      0    enp0s3  
192.168.4.0      *              255.255.255.0    U         0  0      0    enp0s8  
user@client:~$
```

Рисунок 12 – Таблица маршрутизации на узле «Client»

Как видим, в таблице отсутствуют записи для сети 172.16.0.0, а значит пакеты в эту сеть будут отправляться узлу, указанному в качестве шлюза по умолчанию (default gateway). В таблице в качестве default gateway указан узел 10.0.2.2.

Строки таблицы, содержащие * в качестве Gateway (как например строка с 192.168.4.0/24) означают, что хосты с адресами для данной сети находятся

непосредственно за указанным интерфейсом. Обработка таких пакетов не предусматривает их передачу промежуточному узлу. Вместо этого производится поиск узла-получателя по имеющемуся IP с помощью ARP-запроса и взаимодействие с ним на канальном уровне. Чтобы добавить такую строку в таблицу, можно воспользоваться командой:

```
route add -net <адрес сети/маска> dev <интерфейс>
```

Если же поле Gateway указан конкретный узел, то пакеты в эту сеть пересылаются ему. Такие строки добавляются командой:

```
route add -net <адрес сети/маска> gw <адрес шлюза>
```

То есть, при текущих правилах маршрутизации, пакет к 172.16.5.2 согласно строке №1 пойдет на 10.0.2.2, взаимодействие с которым в свою очередь устанавливается строкой №2.

Для того, чтобы пакет шел в нужном направлении, пошлем его на узел Server, который расположен по адресу 192.168.4.1. Для этого добавим правила маршрутизации, указанные на рисунке 13.

```
user@client:~$ sudo route add -net 172.16.0.0/16 gw 192.168.4.1
user@client:~$ netstat -r
Kernel IP routing table
Destination  Gateway      Genmask      Flags MSS Window  irtt Iface
default      10.0.2.2     0.0.0.0      UG    0 0      0 enp0s3
10.0.2.0     *            255.255.255.0 U    0 0      0 enp0s3
172.16.0.0   192.168.4.1 255.255.0.0  UG    0 0      0 enp0s8
192.168.4.0  *            255.255.255.0 U    0 0      0 enp0s8
```

Рисунок 13 – Добавление нового правила маршрутизации

В случае, если необходимо удалить правило из таблицы, используется та же команда *route*, но *add* заменяется на *del*.

Чтобы удостовериться в том, что Client действительно направляет пакеты к 172.16.0.0 через Server, запустим ping 172.16.5.2 на узле Client

(рисунок 14) и будет прослушивать интерфейс `enp0s8` на узле `Server` с помощью `tcpdump` (рисунок 15).

```
user@client:~$ ping 172.16.5.2
PING 172.16.5.2 (172.16.5.2) 56(84) bytes of data.
From 192.168.4.1 icmp_seq=1 Destination Host Unreachable
From 192.168.4.1 icmp_seq=2 Destination Host Unreachable
From 192.168.4.1 icmp_seq=3 Destination Host Unreachable
```

Рисунок 14 – Отправка пинг-запроса с узла `Client` на `172.16.5.2`

```
user@server:~$ sudo tcpdump -i enp0s8
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp0s8, link-type EN10MB (Ethernet), capture size 262144 bytes
18:21:13.725027 IP 192.168.4.2 > 172.16.5.2: ICMP echo request, id 1412, seq 1, length 64
18:21:14.730854 IP 192.168.4.2 > 172.16.5.2: ICMP echo request, id 1412, seq 2, length 64
18:21:15.735217 IP 192.168.4.2 > 172.16.5.2: ICMP echo request, id 1412, seq 3, length 64
```

Рисунок 15 – Прослушивание интерфейса `enp0s8` на сервере

Команда `tcpdump` позволяет прослушивать сетевой трафик на интерфейсе, указанном с помощью опции `-i`.

На данном этапе `Server` принимает пакеты, которые `Client` посылает на `Web-server`. Чтобы `Server` осуществлял пересылку трафика (`IP forwarding`), необходимо включить соответствующую функцию. Для этого откроем файл `/etc/sysctl.conf` в текстовом редакторе `nano`:

```
sudo nano /etc/sysctl.conf
```

Найдем строку `net.ipv4.ip_forward`, раскомментируем её и установим данный параметр в значение `1`.

Сохраним файл (в редакторе `nano` это осуществляется сочетанием клавиш `Ctrl+O`) и применим новые настройки командой:

```
sudo sysctl -p
```

Теперь снова запустим пинг с клиента, как показано на рисунке 14, но на этот раз на сервере будем прослушивать интерфейс `enp0s9`, чтобы удостовериться в том, что трафик пересылается с интерфейса `enp0s8` на `enp0s9`. Вывод `tcpdump` показан на рисунке 16.

```
user@server:~$ sudo tcpdump -i enp0s9
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp0s9, link-type EN10MB (Ethernet), capture size 262144 bytes
18:46:26.469966 IP 192.168.4.2 > 172.16.5.2: ICMP echo request, id 1417, seq 1, length 64
18:46:27.479915 IP 192.168.4.2 > 172.16.5.2: ICMP echo request, id 1417, seq 2, length 64
18:46:28.481103 IP 192.168.4.2 > 172.16.5.2: ICMP echo request, id 1417, seq 3, length 64
```

Рисунок 16 – Прослушивание пересылаемых пакетов на интерфейсе `enp0s9`

Видим, что пакеты пересылаются, но ответов на них не последовало.

На этот раз узел Web-server не знает, куда пересылать ответы для 192.168.4.2.

4. Настройка трансляции адресов

В предыдущем разделе была настроена отправка пакетов к 172.16.5.2. Для обратной отправки можно было бы добавить правила маршрутизации на узел Web-server, но в реальной ситуации этот узел может не быть доступным для конфигурирования (например, если это веб-сервер в интернете).

В таком случае можно настроить узел Server для трансляции IP-адресов (MASQUERADING). При включенной трансляции узел Server будет заменять IP-адрес перенаправляемых пакетов на свой. При получении ответа будет производиться обратная замена.

Для настройки трансляции адресов воспользуемся набором утилит *iptables*.

Трансляция адресов (MASQUERADING) задается следующей командой:

```
iptables -t nat -A POSTROUTING -o <iface> -s <net ip/mask> -j MASQUERADE
```

где *<iface>* – интерфейс, на котором осуществляется замена IP,

<net ip/mask> – сеть, адреса которой необходимо заменять.

Настроим трансляцию адресов на узле Server, запустим пинг с клиента и будет прослушивать интерфейс сервера enp0s9 (рисунок 17).

```
user@server:~$ sudo iptables -t nat -A POSTROUTING -o enp0s9 -s 192.168.4.0/24 -j MASQUERADE
user@server:~$ sudo tcpdump -i enp0s9
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp0s9, link-type EN10MB (Ethernet), capture size 262144 bytes
19:09:39.727059 IP 172.16.5.1 > 172.16.5.2: ICMP echo request, id 1419, seq 5, length 64
19:09:39.730586 IP 172.16.5.2 > 172.16.5.1: ICMP echo reply, id 1419, seq 5, length 64
19:09:40.731222 IP 172.16.5.1 > 172.16.5.2: ICMP echo request, id 1419, seq 6, length 64
19:09:40.734109 IP 172.16.5.2 > 172.16.5.1: ICMP echo reply, id 1419, seq 6, length 64
```

Рисунок 17 – Осуществление замены IP-адресов

Как видим, теперь в качестве адреса отправителя указан 172.16.5.1, что позволяет веб-серверу отвечать на этот адрес. Теперь пинг-запрос проходит (рисунок 18).

```
user@client:~$ ping 172.16.5.2
PING 172.16.5.2 (172.16.5.2) 56(84) bytes of data.
64 bytes from 172.16.5.2: icmp_seq=1 ttl=63 time=0.953 ms
64 bytes from 172.16.5.2: icmp_seq=2 ttl=63 time=3.55 ms
```

Рисунок 18 – Вывод команды ping после настройки трансляции адресов

5. Создание OpenVPN-сервера

OpenVPN – это виртуальная приватная сеть, использующая TLS/SSL. Это означает, что OpenVPN использует сертификаты для шифрования трафика между сервером и клиентами. Для выпуска доверенных сертификатов (trusted certificates) нам потребуется создать наш собственный центр сертификации.

Для начала скопируем шаблонную директорию easy-rsa в нашу домашнюю директорию с помощью команды *make-cadir*:

```
make-cadir ~/openvpn-ca
```

Далее зайдём в эту директорию для начала настройки центра сертификации:

```
cd ~/openvpn-ca
```

Для настройки переменных нашего центра сертификации нам необходимо отредактировать файл *vars*. Откройте этот файл в вашем текстовом редакторе:

```
nano vars
```

Внутри файла вы найдёте переменные, которые можно отредактировать, и которые задают параметры сертификатов при их создании. Нам нужно изменить всего несколько переменных.

Перейдите ближе к концу файла и найдите настройки полей, используемые по умолчанию при создании сертификатов. Они должны выглядеть примерно так:

```
export KEY_COUNTRY="US"  
export KEY_PROVINCE="CA"  
export KEY_CITY="SanFrancisco"  
export KEY_ORG="Fort-Funston"  
export KEY_EMAIL="me@myhost.mydomain"  
export KEY_OU="MyOrganizationalUnit"
```

Замените значения, выделенные красным, на что-нибудь другое, не оставляйте их не заполненными:

```
export KEY_COUNTRY="RU"  
export KEY_PROVINCE="ZA"  
export KEY_CITY="Zelenograd"  
export KEY_ORG="MIET"  
export KEY_EMAIL="miet@miet.ru"  
export KEY_OU="IB-21"
```

Пока мы в этом файле, отредактируем значение KEY_NAME чуть ниже, которое заполняет поле субъекта сертификатов. Для простоты зададим ему название server:

```
export KEY_NAME="server"
```

Сохраните и закройте файл.

Теперь мы можем использовать заданные нами переменные и утилиты easy-rsa для создания центра сертификации.

Убедитесь, что вы находитесь в директории центра сертификации и используйте команду *source* к файлу *vars*:

```
cd ~/openvpn-ca  
source vars
```

Вы должны увидеть следующий вывод:

```
NOTE: If you run ./clean-all, I will be doing a rm -rf on /home/sammy/openvpn-ca/keys
```

Убедимся, что мы работаем в "чистой среде" выполнив следующую команду:

```
./clean-all
```

Теперь мы можем создать наш корневой центр сертификации командой:

```
./build-ca
```

Эта команда запустит процесс создания ключа и сертификата корневого центра сертификации. Поскольку мы задали все переменные в файле *vars*, все необходимые значения будут введены автоматически. Нажимайте ENTER для подтверждения выбора

Теперь у нас есть центр сертификации, который мы сможем использовать для создания всех остальных необходимых нам файлов.

Далее создадим сертификат, пару ключей и некоторые дополнительные файлы, используемые для осуществления шифрования, для нашего сервера.

Начнём с создания сертификата OpenVPN и ключей для сервера. Это можно сделать следующей командой:

```
./build-key-server server
```

Вывод опять будет содержать значения по умолчанию, переданные этой команде (*server*), а также значения из файла *vars*.

Согласитесь со всеми значениями по умолчанию, нажимая ENTER. Не задавайте *challenge password*. В конце процесса два раза введите *y* для подписи и подтверждения создания сертификата

Далее создадим оставшиеся файлы. Мы можем сгенерировать сильные ключи протокола Диффи-Хеллмана, используемые при обмене ключами, командой:

```
./build-dh
```

Для завершения этой команды может потребоваться несколько минут.

Далее мы можем сгенерировать подпись HMAC для усиления способности сервера проверять целостность TLS:

```
openvpn --genkey --secret keys/ta.key
```

Далее мы можем сгенерировать сертификат и пару ключей для клиента. Вообще это можно сделать и на клиентской машине и затем подписать полученный ключ центром сертификации сервера, но в этой статье для простоты мы сгенерируем подписанный ключ на сервере.

В этой статье мы создадим ключ и сертификат только для одного клиента. Если у вас несколько клиентов, вы можете повторять этот процесс сколько угодно раз. Просто каждый раз передавайте уникальное значение скрипту.

Поскольку мы можем вернуться к этому шагу позже, мы повторим команду *source* для файла *vars*. Мы будем использовать параметр *client1* для создания первого сертификата и ключа.

Для создания файлов без пароля для облегчения автоматических соединений используйте команду *build-key*:

```
cd ~/openvpn-ca
source vars
./build-key client1
```

Для создания файлов, защищённых паролем, используйте команду *build-key-pass*:

```
cd ~/openvpn-ca
source vars
./build-key-pass client1
```

В ходе процесса создания файлов все значения по умолчанию будут введены, вы можете нажимать ENTER. Не задавайте challenge password и введите у на запросы о подписи и подтверждении создания сертификата.

Далее настроим сервис OpenVPN с использованием созданных ранее файлов.

Нам необходимо скопировать нужные нам файлы в директорию */etc/openvpn*.

Сначала скопируем созданные нами файлы. Они находятся в директории *~/openvpn-ca/keys*, в которой они и были созданы. Нам необходимо скопировать сертификат и ключ центра сертификации, сертификат и ключ сервера, подпись HMAC и файл Diffie-Hellman:

```
cd ~/openvpn-ca/keys
sudo cp ca.crt ca.key server.crt server.key ta.key dh2048.pem /etc/openvpn
```

Далее нам необходимо скопировать и распаковать файл-пример конфигурации OpenVPN в конфигурационную директорию, мы будем использовать этот файл в качестве базы для наших настроек:

```
gunzip -c /usr/share/doc/openvpn/examples/sample-config-files/server.conf.gz | sudo tee /etc/openvpn/server.conf
```

Теперь, когда наши файлы находятся на своём месте, займёмся настройкой конфигурационного файла сервера:

```
sudo nano /etc/openvpn/server.conf
```

Сначала найдём секцию HMAC поиском директивы *tls-auth*. Удалите ";" для того, чтобы раскомментировать строку с *tls-auth*. Далее добавьте параметр *key-direction* и установите его значение в "0":

```
tls-auth ta.key 0 # This file is secret
key-direction 0
```

Далее найдём секцию шифрования, нас интересуют закомментированные строки *cipher*. Шифр AES-128-CBC обеспечивает хороший уровень шифрования и широко поддерживается другими программными продуктами. Удалите ";" для раскомментирования строки AES-128-CBC:

```
cipher AES-128-CBC
```

Под этой строкой добавьте строку *auth* и выберите алгоритм HMAC. Хорошим выбором будет SHA256:

```
auth SHA256
```

Наконец, найдите настройки *user* и *group* и удалите ";" для раскомментирования этих строк:

```
user nobody
group nogroup
```

Мы готовы включить сервис OpenVPN на нашем сервере. Мы можем сделать это с помощью *systemd*.

Нам необходимо запустить сервер OpenVPN указав имя нашего файла конфигурации в качестве переменной после имени файла *systemd*. Файл

конфигурации для нашего сервера называется `/etc/openvpn/server.conf`, поэтому мы добавим `@server` в конец имени файла при его вызове:

```
sudo systemctl start openvpn@server
```

Убедимся, что сервис успешно запущен командой:

```
sudo systemctl status openvpn@server
```

Вы также можете проверить доступность интерфейса OpenVPN `tun0` следующей командой:

```
ip addr show tun0
```

Если всё в порядке, настроим сервис на автоматическое включение при загрузке сервера:

```
sudo systemctl enable openvpn@server
```

6. Создание конфигурации OpenVPN-клиента

Настроим на сервере систему для простого создания файлов конфигурации для клиентов.

В домашней директории создайте структуру директорий для хранения файлов:

```
mkdir -p ~/client-configs/files
```

Поскольку наши файлы конфигурации будут содержать клиентские ключи, мы должны настроить права доступа для созданных директорий:

```
chmod 700 ~/client-configs/files
```

Далее скопируем конфигурацию-пример в нашу директорию для использования в качестве нашей базовой конфигурации:

```
cp /usr/share/doc/openvpn/examples/sample-config-files/client.conf ~/client-configs/base.conf
```

Откройте этот файл в вашем текстовом редакторе:

```
nano ~/client-configs/base.conf
```

Сделаем несколько изменений в этом файле.

Сначала найдите директиву *remote*. Эта директива сообщает клиенту адрес нашего сервера OpenVPN. Это должен быть публичный IP адрес вашего сервера OpenVPN.

```
# The hostname/IP and port of the server.  
# You can have multiple remote entries  
# to load balance between the servers.  
remote server_IP_address 1194
```

Далее раскомментируйте директивы *user* и *group* удаляя ";" :

```
# Downgrade privileges after initialization (non-Windows only)  
user nobody  
group nogroup
```

Найдите директивы *ca*, *cert* и *key*. Закомментируйте эти директивы, так как мы будем добавлять сертификаты и ключи в самом файле:

```
# SSL/TLS parms.  
# See the server config file for more  
# description. It's best to use  
# a separate .crt/.key file pair  
# for each client. A single ca  
# file can be used for all clients.  
#ca ca.crt  
#cert client.crt  
#key client.key
```

Добавьте настройки *cipher* и *auth* согласно заданным в файле */etc/openvpn/server.conf*:

```
cipher AES-128-CBC  
auth SHA256
```

Далее добавьте директиву *key-direction* в любое место в файле. Она должна иметь значение "1" для корректной работы сервера:

```
key-direction 1
```

Наконец, добавьте несколько закомментированных строк. Мы ходим добавить эти строки в каждый файл конфигурации, но они будут включены только для клиентов на Linux, которые используют файл */etc/openvpn/update-resolv-conf*. Этот скрипт использует утилиту *resolvconf* для обновления информации DNS на клиентах Linux.

```
# script-security 2
# up /etc/openvpn/update-resolv-conf
# down /etc/openvpn/update-resolv-conf
```

Если ваш клиент работает на Linux и использует файл */etc/openvpn/update-resolv-conf*, вы должны раскомментировать эти строки в сгенерированном клиентском файле конфигурации OpenVPN.

Сохраните и закройте файл.

Теперь создадим простой скрипт для генерации файлов конфигурации с релевантными сертификатами, ключами и файлами шифрования. Он будет помещать сгенерированные файлы конфигурации в директорию *~/client-configs/files*.

Создайте и откройте файл *make_config.sh* внутри директории *~/client-configs*:

```
nano ~/client-configs/make_config.sh
```

Вставьте в этот файл код из приложения А.

Сохраните и закройте файл.

Сделайте его исполняемым файлом командой:

```
chmod 700 ~/client-configs/make_config.sh
```

Теперь мы можем легко сгенерировать файлы конфигурации клиентов.

Ранее был создан сертификат *client1.crt* и ключ клиента *client1.key* командой. */build-key client1*. Вы можете сгенерировать конфигурацию для этих файлов перейдя в директорию *~/client-configs* и используя только что созданный скрипт:

```
cd ~/client-configs
./make_config.sh client1
```

Если всё прошло успешно, мы должны получить файл *client1.ovpn* в директории *~/client-configs/files*:

```
ls ~/client-configs/files
```

```
Вывод: client1.ovpn
```

Теперь мы должны переместить файл конфигурации на клиентское устройство.

На клиентском устройстве проверьте, содержит ли ваш дистрибутив скрипт */etc/openvpn/update-resolv-conf*:

```
ls /etc/openvpn
```

```
Вывод:
```

```
update-resolve-conf
```

Далее отредактируйте полученный с сервера файл конфигурации клиента OpenVPN:

```
nano client1.ovpn
```

Если вам удалось найти файл *update-resolv-conf*, раскомментируйте следующие строки файла:

```
script-security 2
up /etc/openvpn/update-resolv-conf
down /etc/openvpn/update-resolv-conf
```

Сохраните и закройте файл.

Теперь вы можете соединиться с VPN используя команду *openvpn* следующим образом:

```
sudo openvpn --config client1.ovpn
```

В результате вы подключитесь к серверу.

7. Перенаправление трафика в защищенный туннель

После создания сетевого интерфейса (туннеля) с именем *tun0* на виртуальных машинах *Client* и *Server*, необходимо настроить правила маршрутизации таким образом, чтобы весь трафик от клиента перенаправлялся через данный интерфейс. Производится это с помощью команд, описанных в части 3, 4.

Вначале необходимо удостовериться, что между *Client* и *Server* существует связь помимо туннеля. Если весь трафик (в том числе уже туннелированный) будет перенаправляться в туннель, то соединение установить не удастся.

Далее, необходимо удостовериться, что проходит пинг-запрос с клиента на сервер с использованием адресов туннеля.

Затем на клиентской машине добавляются соответствующие правила маршрутизации, а на сервере – правила трансляции IP-адресов.

Составьте отчет о выполнении практической работы.

Включите в него копии экрана пошагового выполнения задания и ответы на вопросы практической работы.

Код скрипта генерации конфигурационного файла клиента

```
#!/bin/bash

# First argument: Client identifier

KEY_DIR=~/.openvpn-ca/keys
OUTPUT_DIR=~/.client-configs/files
BASE_CONFIG=~/.client-configs/base.conf

cat ${BASE_CONFIG} \
  <(echo -e '<ca>') \
  ${KEY_DIR}/ca.crt \
  <(echo -e '</ca>\n<cert>') \
  ${KEY_DIR}/${1}.crt \
  <(echo -e '</cert>\n<key>') \
  ${KEY_DIR}/${1}.key \
  <(echo -e '</key>\n<tls-auth>') \
  ${KEY_DIR}/ta.key \
  <(echo -e '</tls-auth>') \
  > ${OUTPUT_DIR}/${1}.ovpn
```